Personal Reflection - James Huang

Throughout the project, I was responsible for the maximal-matching 2-approximation algorithm for the minimum vertex cover problem. I contributed by overseeing AI-assisted development and implementation, personally writing and formatting report sections, requesting additional relabelling and benchmark consistency tests, rewriting my presentation script, and creating the worked-example slides. I used ChatGPT and Codex for explanations, implementation and test design, and feedback on the technical accuracy of my report, script, and slides. This support required me to review the outputs and develop my understanding rather than assume that functioning code was sufficient.

My understanding developed through questions about the problem and its representation. Initially, I needed clarification that a vertex cover selects vertices touching every edge, rather than selecting edges themselves. I also questioned whether straight-line examples meant the implementation only supported paths. Learning that connections define the graph, independently of its drawing, helped me interpret other graph families and construct presentation examples.

The four-vertex path clarified the algorithm's worst-case solution quality. With sorted edges (0,1), (1,2), and (2,3), it returns four vertices although {1,2} is an optimum cover, reaching the factor-two bound. I learned that this guarantee is multiplicative, not an allowance of two additional vertices. Runtime is a separate issue: the implementation takes $O(n + m)$ time because it initialises a vertex array and scans every edge. Dense inputs such as complete graphs contain $\Theta(n^2)$ edges, making the scan quadratic in the number of vertices, although still linear in the graph's represented size.

The design decision I found hardest to understand was why an edge must be accepted only when both endpoints are unmatched. While rewriting my script, I spent the most time on the two invariants of my algorithm: selected matching edges have disjoint endpoints, and every processed edge has a marked endpoint. These connected the code to the proof. Disjoint edges require distinct covering vertices, establishing a lower bound, while permanent marks ensure that processed edges remain covered.

Through creating an incorrect worked example, I strengthened this understanding. For the path 0–1–2, incorrectly accepting both edges produces overlapping selected edges. Their count is two, although vertex 1 alone covers the graph, so the claimed lower bound fails. I created a slide highlighting the shared endpoint and refined its labels to distinguish the incorrect modification from the actual algorithm. This made the condition meaningful rather than something I simply repeated.

Working alongside Derek helped me understand how our algorithms needed to support a consistent comparison. His exact algorithm aims to prove optimality, while my approximation guarantees a feasible cover within twice the optimum. Using the same graph representation made their outputs comparable, but I learned that an interrupted exact result cannot automatically be treated as the optimum when assessing my algorithm's quality. I also requested broader relabelling and benchmark consistency tests to strengthen confidence in our shared experiment workflow. This showed me that my responsibility extended beyond explaining my own algorithm: I also needed to understand how its results could be compared and interpreted alongside my partner's work.

Next time, I would trace examples and record invariants before requesting implementation assistance. I would keep notes about misunderstandings, corrections, and decisions throughout development, and establish expected test outcomes beforehand. Throughout the project, I learned to assess an algorithm through its assumptions, guarantees, and evidence. Rewriting explanations and building diagrams helped me move from recognising its steps to explaining why those steps are necessary and what would fail if they changed.